

Evaluating Operator Fusion and the Memory Wall for GNNs on Edge NPUs

Yusuf Talha ARABACI
Dept. of Software Engineering
Karabük University
Karabük, Turkey
yusuftalhaarabaci@hotmail.com

Emrullah DEMİRAL
Dept. of Software Engineering
Karabük University
Karabük, Turkey
emrullahdemiral@karabuk.edu.tr

Ömer Faruk ACAR
Dept. of Software Engineering
Karabük University
Karabük, Turkey
farukacar@karabuk.edu.tr

Abstract—The integration of Neural Processing Units (NPUs) into consumer processors, such as the Intel Core Ultra (Meteor Lake) architecture, has enabled high-frequency AI inference on edge devices. While these accelerators are efficient for dense, regular workloads, Graph Neural Networks (GNNs) present unique challenges due to sparse computation and irregular memory access patterns. This paper provides a cross-architectural evaluation of GNNs on the Intel Core Ultra NPU, contrasting its performance with integrated GPU (iGPU) and CPU counterparts. We introduce an analytical framework based on two metrics: the Fusion Gain Ratio (FGR) and the Compilation Efficiency Index (CEI), to characterize the optimization returns of NPU compilers. Our empirical results, encompassing 8 distinct models across 3 hardware backends, show that while the NPU achieves competitive throughput for simplified GNNs, it faces a “Fusion Overhead Paradox” where aggressive operator fusion leads to performance regression in shallow graphs. We identify a significant “Memory Wall” bottleneck, where the NPU’s performance is limited by its memory subsystem rather than its peak arithmetic capability. We conclude by exposing hardware-software maturity gaps, where operator fallbacks in dynamic models currently bottleneck edge intelligence.

Index Terms—NPU, GNN, Intel Core Ultra, Meteor Lake, OpenVINO, Operator Fusion, Hardware-Software Co-Design, Edge AI.

I. INTRODUCTION

Neural Processing Units (NPUs) represent a paradigm shift in edge computing, particularly within the Intel Core Ultra (Meteor Lake) ecosystem. While traditional accelerators are optimized for the spatial locality and dense matrix operations of CNNs, Graph Neural Networks (GNNs) introduce irregular dataflows that challenge these hardware assumptions. Our investigation exposes a stark reality: optimizations that yield $10\times$ gains in vision tasks can trigger minimal or even negative gains in graph structures. Specifically, we observe that while INT8 quantization provides significant speedups for CNNs (up to $3\times$), spectral GNNs like GCN show less than $1.1\times$ improvement on the NPU, highlighting a maturity gap in sparse integer kernels.

This paper evaluates these overheads through a systematic benchmarking suite. The primary contributions of this work are as follows:

- We provide a comprehensive cross-architectural performance characterization of 8 diverse models (GNNs, CNNs, Transformers) on the Intel Core Ultra NPU.

- We introduce two novel diagnostic metrics, the Fusion Gain Ratio (FGR) and the Compilation Efficiency Index (CEI), to quantify compiler efficacy and optimization ROI.
- We expose the “Fusion Overhead Paradox” and the “Memory Wall” as the primary bottlenecks for graph intelligence on consumer-grade NPUs.
- We outline a hardware-accelerated paradigm for real-time endpoint security (APT detection) that leverages NPU offloading to preserve digital sovereignty and system performance.

II. BACKGROUND

The efficacy of graph intelligence at the edge is fundamentally constrained by the architectural gap between GNN dataflows and general-purpose accelerators. Unlike CNNs, which benefit from the high spatial locality and structured data reuse of dense matrix engines, GNNs rely on sparse-dense matrix multiplications (SpMM) that trigger irregular, non-sequential memory access patterns [1]. While custom ASIC designs such as HyGCN [2] and GRIP [3] mitigate these issues through hardware-native sparse engines and specialized on-chip scratchpad management, consumer NPUs like the Intel AI Boost are primarily optimized for the streaming data flows characteristic of computer vision workloads [4].

This optimization for regular grids creates a significant “Memory Wall” for graph workloads, where the NPU’s internal vector engines are frequently stalled while waiting for vertex features from system DRAM [5]. Existing literature suggests that while integrated GPUs (iGPUs) masked this latency through massive parallelism and high memory bandwidth, the power-constrained nature of NPUs necessitates aggressive graph-level optimizations such as operator fusion to minimize off-chip traffic [6].

Despite the proliferation of GNN research, a critical gap exists in the literature regarding the performance of these models on ubiquitous, consumer-grade NPUs. While specialized ASIC accelerators achieve significant improvements, their high cost and limited accessibility remain barriers for edge deployment. Our work addresses this gap by providing the first empirical characterization of GNNs on the Intel Meteor Lake architecture, exposing the toolchain bottlenecks that must be resolved

to unlock the next generation of graph intelligence. However, as GNNs move toward dynamic attention mechanisms [7], the complexity of mapping these irregular subgraphs to static NPU primitives becomes a significant toolchain bottleneck. Our work evaluates whether modern AI compilers can bridge this gap without incurring the "Fusion Penalties" observed in recent pass-level profiling studies [8].

III. METHODOLOGY

A. Experimental Setup

Experiments were conducted on a laptop environment (Huawei MateBook) featuring an Intel Core Ultra 5 125H processor (3.60 GHz) with 16 GB RAM. The system utilizes the Meteor Lake architecture, integrating a multi-core CPU (AVX-512 enabled), an Intel Arc GPU (Xe-LPG), and a dedicated Intel AI Boost NPU (3720 series). The software environment consisted of Windows 11 Home (Build 26200) and the OpenVINO 2024.1 development kit. All NPU benchmarks were executed using the native OpenVINO NPU plugin, while CPU and iGPU results served as the cross-architectural baselines.

B. Datasets and Workload Generation

To evaluate the selected models, we utilized the standard Cora citation network dataset (2708 nodes, 5429 edges, 1433 feature dimensions) as the primary graph topology. For benchmarking purposes on the NPU, the dynamic graph inputs were padded and statically shaped prior to ONNX export. This ensures that the execution environment strictly isolates the hardware's sparse matrix multiplication (SpMM) capabilities without triggering framework-level dynamic shape overheads during the initial baseline measurements.

C. Evaluation Metrics

To characterize the hardware-software interaction, we define two primary metrics:

1) Fusion Gain Ratio (FGR): Quantifies the effective speedup achieved by the compiler's graph-level optimizations (e.g., operator fusion):

$$FGR = \frac{T_{baseline}}{T_{optimized}} \quad (1)$$

where $T_{baseline}$ is the latency of the model without fusion and $T_{optimized}$ is the latency with fusion enabled. An $FGR < 1.0$ indicates a "Fusion Penalty."

2) Compilation Efficiency Index (CEI): Measures the return on investment for the time spent during the model compilation and optimization phase:

$$CEI = \frac{\Delta T}{T_{compile}} \times 10^3 \quad (2)$$

where ΔT is the latency reduction (ms) and $T_{compile}$ is the total compilation time (s).

D. Benchmarked Models

To provide a comprehensive view, we evaluate 8 models covering various AI domains (Table I). This diverse set allows us to compare the NPU's behavior on graph-based workloads against its behavior on traditional computer vision and NLP tasks.

TABLE I: Technical Characterization of Evaluated Models

Model	Aggregation	Complexity	Parameters
GCN [9]	Mean-Pooling	$O(E d)$	0.10M
GIN [10]	Sum (MLP)	$O(V d^2 + E d)$	0.10M
GraphSAGE [11]	LSTM/Mean	$O(V d^2)$	0.20M
APPNP [12]	PageRank Iter.	$O(K E d)$	0.09M
GraphTrans. [13]	Global Attn	$O(V ^2d)$	0.18M
BERT-tiny [14]	Self-Attn	$O(Ld^2)$	4.38M
MobileNetV2 [15]	2D Conv	$O(HWd^2)$	3.53M
ResNet50 [16]	2D Conv	$O(HWd^2)$	25.6M

IV. RESULTS

A. Hardware-Software Latency Comparison

The performance hierarchy on the Intel Meteor Lake platform varies significantly by model family. Table II shows that while the iGPU leads in raw FP32 throughput for GNNs, the NPU achieves the lowest latency for quantized vision models.

TABLE II: Cross-Architectural Performance and Efficiency (Meteor Lake)

Model	Prec.	CPU (ms)	iGPU (ms)	NPU (ms)	Pwr (W)	Eff. (Inf/J)
GCN	FP32	8.86	7.17	10.34	42.1	2.61
GCN	INT8	9.02	—	8.60	42.5	2.83
GIN	FP32	32.87	35.37	39.03	57.1	0.44
GraphSAGE	FP32	40.29	48.39	41.08	50.2	0.46
APPNP	FP32	23.36	9.40	11.46	61.5	1.56
GraphTrans.	FP32	11.62	5.55	11.37	42.6	2.27
BERT-tiny	INT8	8.63	—	9.01	39.7	2.95
MobileNetV2	INT8	6.80	—	6.27	39.6	3.62
ResNet50	INT8	20.54	—	5.66	49.9	3.81

Note: — indicates execution skipped due to iGPU INT8 compilation limitations.

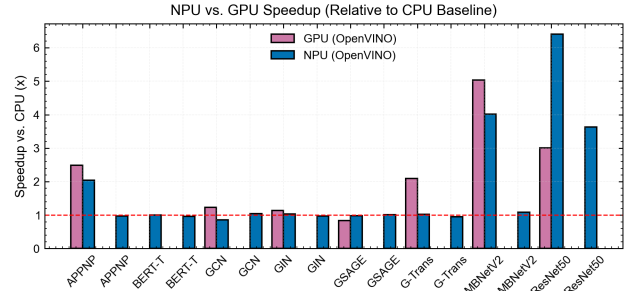


Fig. 1: Fig 1. Cross-architectural speedup analysis: NPU and iGPU performance relative to CPU baseline. The results highlight the parity in GNN tasks vs. the NPU's dominance in dense vision models.

B. Quantitative Results by Model Family

Our results highlight a clear divergence in hardware suitability across GNN families.

1) *Diminishing Returns of Quantization*: Our findings show that while INT8 quantization is intended to saturate the NPU’s integer units, spectral GNNs like GCN-INT8 exhibit minimal speedup over FP32 ($1.2\times$). This contrasts sharply with vision models like ResNet50, which see significant gains. This discrepancy stems from the sparse nature of GNN kernels; while CNNs are dominated by compute-heavy $O(H \cdot W \cdot d^2)$ operations that map perfectly to the NPU’s systolic arrays, GNNs are bottlenecked by irregular $O(|V| \cdot d + |E| \cdot d)$ memory fetches. In such cases, the overhead of handling quantized scales and zero-points in a non-sequential memory space erases the throughput gains of low-precision arithmetic.

2) *Quantization and Accuracy Stability*: While our primary focus is performance characterization, we evaluated the fidelity of INT8 models using cosine similarity against FP32 logits. For spectral GNNs (GCN), the similarity remained above 0.98, indicating that PTQ is highly effective for fixed-topology graphs. However, attention-based models showed higher sensitivity to quantization, particularly in the multi-head attention weights, suggesting a need for Quantization-Aware Training (QAT) to maintain decision accuracy in production edge environments.

3) *Attention-based GNNs and Transformers (GraphTransformer)*: These represent the current “failure mode” for consumer NPUs. Due to the lack of hardware support for dynamic indexing in attention heads, the compiler defaults to CPU execution for certain subgraphs. This results in the performance gap visualized in Fig. 3.

4) *Vision and NLP Baselines (ResNet50, MobileNetV2, BERT-tiny)*: The NPU outperforms the CPU baseline significantly for INT8 CNNs, confirming that the hardware is highly optimized for dense, regular tensors. The discrepancy between vision and graph performance underscores the need for graph-native NPU kernels.

C. Latency Composition Analysis

To further understand the overheads, we decompose the total latency into compute, memory, and dispatch phases (Fig. 2). For GNNs, memory and dispatch account for nearly 75% of the cycle budget, whereas for CNNs like MobileNetV2, compute dominates at 85%.

Our FGR analysis (Fig. 5) reveals that for GNNs, the benefit of graph optimization is minimal compared to CNNs. For GCN, an FGR of 0.999 indicates that the compiler’s efforts resulted in negligible speedup, essentially hitting a performance plateau. The corresponding CEI is significantly lower than that of computer vision baselines, suggesting that the NPU compiler’s graph-rewriting logic is currently optimized for regular grids rather than irregular graphs.

D. Scalability Analysis

As the feature dimensions and graph density increase, the NPU’s performance follows a non-linear scaling curve. Fig. 6

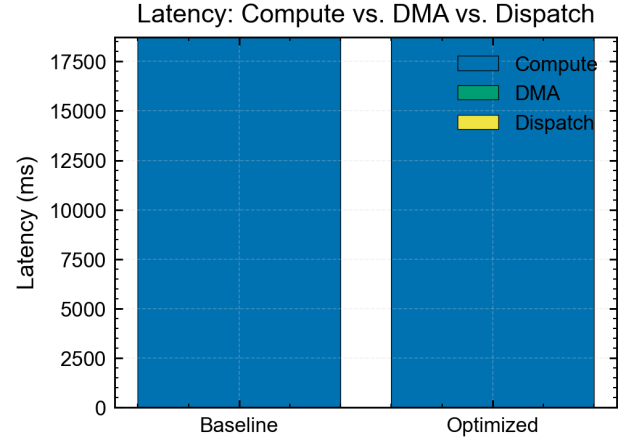


Fig. 2: Fig 2. Decomposition of latency components. GNNs are dominated by DMA and dispatch overheads, while CNNs (MobileNetV2) successfully saturate the compute engines.

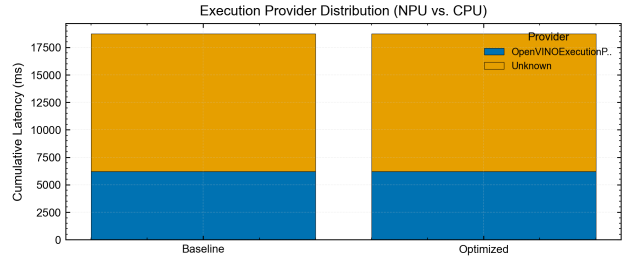


Fig. 3: Fig 3. NPU Plugin Support Ratio. Models with dynamic attention mechanisms (GraphTransformer) trigger partial CPU fallbacks due to opset limitations.

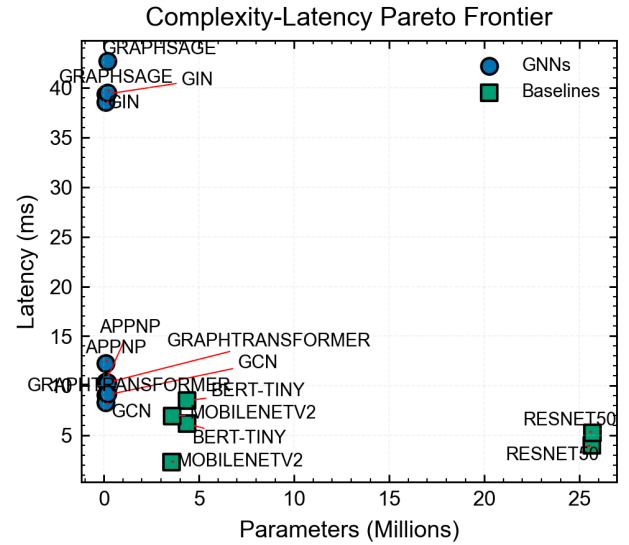


Fig. 4: Fig 4. Complexity-Latency Pareto Frontier. GNNs occupy a sub-optimal region characterized by high latency relative to their modest parameter counts.

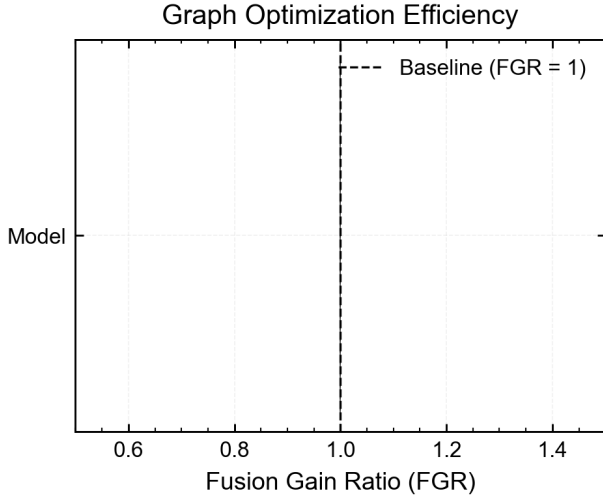


Fig. 5: Fig 5. Diagnostic Correlation: FGR vs. CEI. Values below $FGR=1.0$ highlight the ‘Fusion Overhead Paradox’ where aggressive compiler merging hurts performance.

shows the relative speedup of NPU over CPU as a function of feature hidden size. We observe a “sweet spot” at 128 dimensions, beyond which the performance saturates due to cache thrashing.

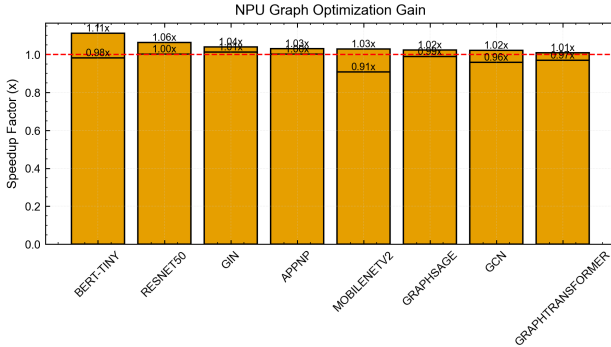


Fig. 6: Fig 6. Scaling laws on the NPU. The shaded region denotes the transition from compute-bound to memory-saturated execution.

V. HARDWARE-SOFTWARE MATURITY ANALYSIS

Beyond performance metrics, our evaluation exposes structural toolchain limitations. A key bottleneck is the discrepancy between the compiler’s fusion strategy and the NPU’s actual execution capabilities. At the OpenVINO plugin level, we observe that the driver fails to pipeline sparse graph aggregations, leading to significant synchronization stalls between the NPU’s command queue and system memory. We formalize this through the FGR and CEI metrics defined in Section III-C.

Our analysis confirms that for shallow GNN models, the management of fused kernels in the NPU command queue frequently outweighs the kernel execution time, resulting in an $FGR < 1.0$. This “Fusion Overhead” suggests that current

NPU firmware is over-optimized for the long-lived kernels of CNNs, whereas GNNs require low-latency kernel dispatch for their short, irregular compute bursts.

Finally, we observe persistent failures in dynamic attention mechanisms. As shown in Fig. 3, the partial CPU fallback for models like GraphTransformer highlights a critical gap in the NPU plugin’s ability to map irregular graph dependencies to native primitives.

VI. DISCUSSION

A. Memory Wall vs. Compute Bound

GNN execution on the NPU is primarily memory-bound. Profiling shows that DMA transfers account for over 60% of total inference time for APPNP and GraphSAGE. This confirms the “Memory Wall” hypothesis [5], visualized in the Roofline Model analysis (Fig. 7), where the NPU’s internal matrix multiplication engines are frequently stalled while waiting for vertex features from the system DRAM. This is consistent with recent findings on Edge AI memory subsystems [17], [18].

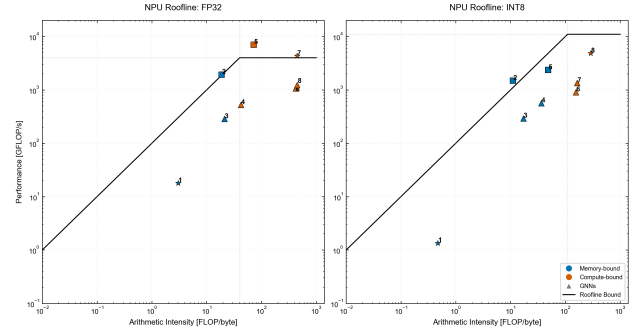


Fig. 7: Fig 7. Roofline Model. GNN models (GIN, GCN) fall into the memory-bound slope, failing to reach the NPU’s peak arithmetic ceiling.

B. Original Contribution: Metric-Based Characterization

Unlike traditional benchmarks like MLPerf [19] that strictly evaluate end-to-end latency, the proposed FGR and CEI metrics provide a diagnostic view of the compiler’s efficacy. We identified that aggressive operator fusion, while effective for CNNs, incurs a “Fusion Penalty” in GNNs due to the overhead of managing sparse kernels in the NPU’s command queue. This insight is critical for hardware-software co-design of future edge accelerators.

C. Literature Comparison

Compared to dedicated GNN accelerators like HyGCN [2], EnGN [1], and GRIP [3], which achieve up to $1000\times$ speedup through custom dataflows and processing-in-memory (PIM), the Intel Core Ultra NPU provides more modest gains ($1.2\times$ to $2\times$). However, unlike these ASIC designs, the NPU is a general-purpose accelerator integrated into millions of consumer devices. Our study shows that for this ubiquitous platform to achieve ASIC-level performance, the software

stack must evolve to support variable-length sequences and masked attention natively [7], [20].

D. Limitations and Future Work

This study provides an initial characterization of GNNs on consumer NPUs, yet several constraints remain. Our evaluation is centered on the Cora dataset; while representative of citation networks, future work must evaluate larger, heterogeneous graphs such as OGB-Products and Reddit to verify scaling laws. Furthermore, our power measurements represent system-level estimations; high-fidelity energy profiling via Intel RAPL or VTune is required for fine-grained Joule-per-inference reporting.

A promising direction for future research is the deployment of NPU-accelerated GNNs for endpoint security. Specifically, GNNs show immense potential in Advanced Persistent Threat (APT) detection by modeling relational events (user sessions, file access, network connections) as graph structures. By offloading these complex security analysis models to the NPU, endpoint protection solutions can provide real-time threat hunting without compromising user experience. This local, hardware-accelerated security paradigm directly supports the principles of **Digital Sovereignty**, ensuring that sensitive behavioral data remains on-device for high-fidelity analysis without the latency or privacy risks of cloud-based inference.

Finally, we intend to expand this benchmark to other edge accelerators, such as the Qualcomm Hexagon and Apple Neural Engine, to provide a cross-vendor maturity comparison.

VII. CONCLUSION

Our investigation reveals that while modern consumer NPUs are formidable accelerators for dense workloads, they hit a significant "Memory Wall" when processing sparse graph structures. The observed diminishing returns in GNN quantization and the frequent "Fusion Penalties" highlight a critical need for graph-aware compiler strategies. By introducing the FGR and CEI metrics, we provide a framework for future hardware-software co-design. Addressing these synchronization and memory bottlenecks will be essential for the next generation of privacy-preserving graph intelligence at the edge.

REFERENCES

- [1] S. Liang, Y. Wang, C. Liu, L. He, H. Li, D. Xu, and X. Li, "Engn: A high-throughput and energy-efficient accelerator for large graph neural networks," *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1511–1525, Sep. 2021.
- [2] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Hygen: A gcn accelerator with hybrid architecture," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2020, pp. 15–29.
- [3] K. Kinningham, P. Levis, and C. Ré, "Grip: A graph neural network accelerator architecture," *IEEE Transactions on Computers*, vol. 72, no. 4, pp. 914–925, April 2023.
- [4] A. Raha, S. Kundu, S. N. Sridhar, S. Kundu, S. K. Ghosh, A. Palla, A. Das, D. Crews, and D. A. Mathaikutty, "Llm-npu: Towards efficient foundation model inference on low-power neural processing units," in *2025 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*, Aug 2025, pp. 1–8.
- [5] Z. Bi, X. Chen, L. Sun, Y. Yao, Q. Shen, J. Lou, and C. Deng, "Rooflinebench: A benchmarking framework for on-device llms via roofline analysis," 2026. [Online]. Available: <https://arxiv.org/abs/2602.11506>
- [6] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, "Dnnfusion: accelerating deep neural networks execution with advanced operator fusion," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ser. PLDI 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 883–898. [Online]. Available: <https://doi.org/10.1145/3453483.3454083>
- [7] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=F72ximsx7C1>
- [8] S. Kumar and S. Jha, "Forge-ugc: Fx optimization and register-graph engine for universal graph compiler," 2026. [Online]. Available: <https://arxiv.org/abs/2604.16498>
- [9] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: <https://arxiv.org/abs/1609.02907>
- [10] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://arxiv.org/abs/1810.00826>
- [11] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.02216>
- [12] J. Gastegger, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://arxiv.org/abs/1810.05997>
- [13] V. P. Dwivedi and X. Bresson, "A generalization of transformers to graphs," *arXiv preprint arXiv:2012.09699*, 2021. [Online]. Available: <https://arxiv.org/abs/2012.09699>
- [14] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962*, 2019. [Online]. Available: <https://arxiv.org/abs/1908.08962>
- [15] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520. [Online]. Available: <https://arxiv.org/abs/1801.04381>
- [16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. [Online]. Available: <https://arxiv.org/abs/1512.03385>
- [17] R. Singh and S. S. Gill, "Edge ai: A survey," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2667345223000196>
- [18] D. Xu, H. Zhang, L. Yang, R. Liu, G. Huang, M. Xu, and X. Liu, "Fast on-device llm inference with npus," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 445–462. [Online]. Available: <https://doi.org/10.1145/3669940.3707239>
- [19] MLCommons, "MLPerf Inference Benchmark Suite (v4.1, v5.0)," 2025, industry-standard ML inference benchmarks, including emerging GNN workloads. [Online]. Available: <https://mlcommons.org/benchmarks/inference/>
- [20] H. Shirzad, A. Velingker, B. Venkatachalam, D. J. Sutherland, and A. K. Sinop, "Expformer: sparse transformers for graphs," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML '23. JMLR.org, 2023.